

Supplementary Material

Lineage-Aware Memory Governance: A Derivation-Gated Framework for Privacy-Preserving Column-Level Access Control in Enterprise AI Agents

Venkata Sangaraju¹ and Sudhir Vissa²

¹Independent Researcher (ORCID: 0009-0001-7716-1342)

²SAGE7 AI, Georgetown, Texas, USA (ORCID: 0009-0003-7865-0863)

This document accompanies the IEEE Access submission “*Lineage-Aware Memory Governance: A Derivation-Gated Framework for Privacy-Preserving Column-Level Access Control in Enterprise AI Agents.*” It reproduces, in full, material that the main paper summarises for length: extended related-work discussion (§1), the line-by-line walkthrough of Algorithm 2 (§2; Algorithm 1’s walkthrough and the full proof of Theorem 1 are now given directly in the main paper and are only reproduced here for self-containedness), extended threat descriptions (§4), the unevaluated composite conflict-detector design (§5), and complete per-experiment result tables (§6). All numbering in this document is local to this document; cross-references to the main paper are given as plain section numbers (main paper, §*n*).

I. EXTENDED RELATED WORK

The main paper (§2) summarises related work in one page against six governance dimensions (Table 1). This section reproduces the full per-system discussion.

A. Agent Memory Systems

Du [1] formalises agent memory as a write-manage-read loop with a three-dimensional taxonomy spanning temporal scope, representational substrate, and control policy. MemGPT [2] instantiated the hierarchy by treating the LLM as an OS that pages between a bounded context window and an external store via function calls, demonstrating coherent long-horizon conversations and large-document analysis. *Limitation relative to our work:* governance in MemGPT is implicit—the LLM decides what to write and retrieve—with no lineage tracking at any layer.

Zep [3] introduced a temporal knowledge graph with dual-axis timestamping (event time vs. ingestion time), enabling point-in-time queries. *Limitation:* its graph captures *when* information arrived, not *how* it was derived. A-MEM [4]—a NeurIPS 2025 paper—builds Zettelkasten-style linked notes with dynamic indexing, achieving strong benchmark performance across six foundation models. *Limitation:* linking structure encodes semantic relatedness between notes, not the data-derivation path of a computed result. All three systems represent the state of the art in conversational or task-level memory and are well-suited to remembering facts, past conversations, and user preferences, but none model the provenance of computed analytical results.

B. Governed and Enterprise Memory Architectures

Taheri’s Governed Memory [5] addresses a real enterprise gap—dozens of agents operating with no shared memory and no common governance—through a dual memory model, tiered governance routing, and a closed-loop schema lifecycle, achieving 99.6% fact recall and zero cross-entity leakage across 500 adversarial queries in a 250-event controlled experiment. *Limitation:* its governance operates on *what is stored and who may access it as a whole entry*; it does not track derivation, so it cannot detect that a permitted entry embeds a forbidden column. Lam et al.’s SSGM [6] decouples memory evolution from execution by enforcing consistency verification, temporal decay modelling, and dynamic access control before any memory consolidation. *Limitation:* SSGM’s dynamic access control enforces

agent-topology constraints (who may communicate with whom), not column-level derivation constraints (what data was touched to produce a result)—the two mechanisms are complementary rather than overlapping. Oracle AI Agent Memory [7] provides a commercially deployed governed memory core with fine-grained lifecycle management and role-based access controls. *Limitation:* role assignment controls access to a memory entry as a whole, not to the individual columns that contributed to it. Abaev et al.’s AgentGuardian [8] learns context-aware access-control policies from execution traces to govern which tool calls an agent may issue. *Limitation:* it governs *tool-call* legitimacy at execution time, not the provenance of values already committed to a shared memory store.

C. Column-Level Security and Data Provenance

Apache Ranger, Databricks Unity Catalog, and BigQuery column-level security enforce column-gated access at the *query layer*: they prevent an unauthorised agent from issuing a query that touches a restricted column. Subramaniam and Krishnan’s DePLOI [9] uses NL2SQL task decomposition to synthesise and audit database access-control implementations against natural-language intent, improving the trustworthiness of the source-layer policies themselves. *Limitation relative to our work:* these are all source-layer controls that fire at query time; none address a result that was legitimately computed once, cached, and then served to a different, less-privileged requester. The W3C PROV standard [10] encodes entity-activity-agent relationships for data provenance, and Herschel et al. [11] survey systems that use provenance to track data flow through complex pipelines. Column-level lineage extraction systems such as LINEAGEX [12]—which parses SQL query trees to infer column-level derivation with high coverage and accuracy, demonstrated at ICDE 2025—represent the state of the art in automatic lineage extraction. Zhou et al.’s survey of the LLM×DATA intersection [13] catalogues how data systems and LLMs increasingly interact but does not treat agent memory retrieval as a governance surface in its own right. *Limitation:* the provenance literature focuses on raw database records and ETL pipelines; our AMU schema applies the same column-level-derivation concept to *agent memory entries*, and LINEAGEX or SQLGLOT [14] (used in Experiment 6) are the natural tools for automatically populating AMU lineage fields in production. No prior work applies column-level provenance to agent memory retrieval governance, or uses derivation graphs to gate cross-department memory sharing—this is the precise gap we fill.

D. Privacy and Security in Multi-Agent Systems

El Yagoubi et al.’s AgentLeak [15]—the first full-stack benchmark for privacy leakage in multi-agent LLM systems—shows that 68.8% of leakage occurs through inter-agent channels (messages, shared memory, tool arguments) invisible to output-only auditors, across 4,979 validated execution traces. Naik et al.’s OMNI-LEAK [16] demonstrates that a single indirect prompt injection can compromise several agents and leak sensitive data even with data-access controls in place; Alizadeh et al. [17] show that simple prompt-injection payloads can cause an LLM agent to leak personal data it observed during task execution, and Wang et al.’s taxonomy of prompt-injection threats [18] and Dehghantanha and Homayoun’s SoK of the agentic attack surface [19] both catalogue memory and context poisoning as first-class attack vectors against agents. Nguyen et al.’s survey of multi-agent system security [20] derives a 193-item threat taxonomy from production multi-agent architectures, including threats to shared memory and inter-agent trust. Summers and Wu’s Data Flow Control [21] enforces declarative, tuple-level data-safety policies inside the DBMS query layer itself using provenance monomials—closely related in spirit to our column-level gate, but operating on live query execution rather than on a persistent cross-agent memory cache. Wang et al.’s AuthGraph [22] defends individual agents against prompt-injection by structurally comparing an execution-derived provenance graph against an authorisation graph, reducing attack success rates from ~40% to 1–2% on two benchmarks—the closest prior work to our own in combining a provenance graph with an authorisation check, but scoped to single-agent tool-calling safety rather than cross-department memory retrieval. Naseh et al. [23] show that retrieval-augmented generation systems are vulnerable to stealthy membership-inference attacks that recover whether a specific record was present in a retrieval corpus, a threat that targets what is stored rather than how it was derived. *Limitation relative to our work:* this literature identifies leakage through inter-agent communication channels, prompt-level manipulation of a single agent, and inference over stored content; our work identifies a complementary vector—leakage through a *shared analytical memory store* that persists even when every individual communication link and every individual agent is otherwise correctly secured. The two families of failure modes are independent and both need to be addressed in production; the threat model (main paper, §5) (threat T8) discusses how prompt-injection and inference-style attacks interact with the lineage record itself.

E. Metric-Definition Governance

Semantic drift—divergent definitions of the same KPI across teams—is a documented source of dashboard inconsistency in enterprise BI [24]. Semantic-layer tools (dbt Metrics, Databricks Unity Catalog, Looker LookML) address this for human analysts by centralising metric definitions, and the Open Semantic Interchange initiative [25], launched in September 2025 by Snowflake, Salesforce, dbt Labs, BlackRock, and RelationalAI, is an industry-wide push toward a vendor-neutral metric specification. *Limitation relative to our work:* these tools govern metrics that *humans* define centrally in advance; they provide no mechanism for detecting drift when an autonomous agent computes a KPI on the fly through its own generated query. Our *definition_hash* conflict detector (Algorithm 2) provides a lightweight, retrofittable foundation for automated metric governance in agentic systems that write results outside a pre-registered semantic layer; the extended fuzzy experiment in the extended fuzzy conflict-detection experiment (main paper, §6) characterises where hash-based detection is, and is not, the right granularity.

II. ALGORITHM WALKTHROUGHS

The main paper now gives the full line-by-line walkthrough for Algorithm 1 (Lineage-Gated Retrieve) directly in System Design (main paper, §3.2), including its worst-/best-case complexity derivation, so it is not repeated here; the pseudocode is reproduced below only so that line numbers cited by the proof in §3 remain unambiguous in this standalone document. Algorithm 2 (Lineage-Aware Write with Conflict Detection) is reproduced below with its full walkthrough, since the main paper gives only a summary of it.

Algorithm 1: Lineage-Gated Retrieve Input: *store, metric, dept, fresh* Output: *result; leaked=False guaranteed*

```

1 candidates <- store[metric]           % all stored
   ANUs for this metric
2 permitted <- P(dept)                 % column set
   the department may see
3 blocked <- False
4
5 for each a in REVERSE(candidates): % most-recent-
   first
6   if S(a) not_subset permitted:      % derivation
   touches a forbidden column
7     blocked <- True
8     continue                       % skip this
   candidate; try the next
9   end if
10  return (reused=True, value=a.value, leaked=False,
   blocked=False)
11 end for
12
13 store[metric].append(fresh)
14 return (reused=False, value=fresh.value, leaked=
   False, blocked=blocked)

```

Algorithm 2: Lineage-Aware Write with Conflict Detection Input: *store, AMU a* Output: *conflict bool*

```

1 % store : shared memory dict (mutated in place)
2 % a : new ANU to persist
3 % conflict : True iff a cross-department definition
   conflict is detected
4
5 existing <- store[a.metric]          % all ANUs stored for
   this metric name
6 conflict <- False
7
8 for each e in existing:
9   if e.dept != a.dept:              % cross-department
   pair
10  if H(e) != H(a):                  % definition hashes
   differ -> potential conflict
11    conflict <- True
12    break                          % one conflict is
   sufficient; raise alert
13  end if
14 end if
15 end for
16
17 store[a.metric].append(a)           % always persist;
   conflict is surfaced, not blocked
18 return conflict

```

Line-by-line walkthrough. Line 5 gathers every AMU already stored under the same metric name as the incoming write, regardless of which department produced it. Line 9 restricts the comparison to *cross-department* pairs: two AMUs written by the same department under the same metric name are assumed to share a definition by construction and are not compared. Line 10 compares the cached definition hashes $H(\cdot)$ (Equation 2) rather than the raw lineage graphs, which is what makes the comparison $O(1)$ per pair rather than requiring a graph-isomorphism check. Lines 11–12 raise the conflict flag and exit the loop on the first cross-department hash mismatch found; one divergent definition is sufficient evidence of drift, so

the algorithm does not need to enumerate all of them. Line 17 always persists the new AMU, whether or not a conflict was detected: *the write path never blocks on a conflict*, it only surfaces one for downstream alerting (the threat model (main paper, §5), T2). This design choice trades a small amount of governance strictness for operational safety—a metric-definition disagreement should never cause a legitimate business query to fail outright. *Complexity*: the loop performs at most n hash comparisons, each $O(1)$, giving $O(n)$ worst case (no conflict, full scan) and $O(1)$ best case (conflict found on the first cross-department comparison), matching the empirical measurements in Table 6.

III. FULL PROOF OF THE SENSITIVE-COLUMN SAFETY THEOREM

The main paper (§4) now states Theorem 1, its full proof, and its two corollaries in full (not merely a proof sketch). The complete statements and proof are reproduced below as well, so that this document remains readable independently of the main paper. Equations (1)–(2) define the cached sensitivity tag $S(a)$ and definition hash $H(a)$ referenced throughout.

$$S(a) = \bigcup_{i=1}^k C_i \cap \mathcal{S} \quad (1)$$

$$H(a) = \text{SHA256} \left(\text{sort}(\text{tables}) \parallel \text{sort} \left(\bigcup_i C_i \right) \parallel \emptyset \right)_{[0:12]} \quad (2)$$

Assumption 1 (Complete Lineage). *For every AMU a in the store, $a.\text{lineage}$ records every table, every column, and the filter logic of the computation that produced $a.\text{value}$.*

Theorem 1 (Sensitive-Column Safety). *Under Assumption 1, Algorithm 1 returns no result to department d that was derived from a sensitive column $c \in \mathcal{S}$ with $c \notin P(d)$.*

Remark (Scope of the guarantee). *The gate checks $S(a) \subseteq P(d)$ where $S(a) = (\bigcup_i C_i) \cap \mathcal{S}$ contains only columns in the sensitive registry $\mathcal{S}$. Non-sensitive columns that appear in a department’s blocked list (such as `amount` for Support in Table 1) are not gate-enforced by Algorithm 1 as written. An operator who wishes to gate-enforce non-sensitive columns as well should extend $\mathcal{S}$ to include them, or change the gate condition to $(\bigcup_i C_i) \subseteq P(d)$ at the cost of blocking more memory reuse.*

Proof. This proof is also given, in identical form, in the main paper (§4); it is reproduced here so the Supplementary Material remains self-contained. By inspection, Algorithm 1 has exactly two statements that return a value to the caller: line 10 (memory-served) and line 14 (fallback). Every invocation terminates by reaching one of the two, since the `for` loop either returns early at line 10 or exhausts candidates and falls through to lines 13–14. We show the returned AMU’s derivation touches no forbidden sensitive column on each path.

Case 1: memory-served path (line 10). Reaching line 10 for candidate a requires the guard at line 6 to have evaluated false, i.e. $S(a) \subseteq P(d)$ holds for that a . Unfolding Equation 1, $S(a) = (\bigcup_{i=1}^k C_i) \cap \mathcal{S}$, so $S(a) \subseteq P(d)$ is equivalent to: for every step $s_i \in a.\text{lineage}$ and every column $c \in C_i \cap \mathcal{S}$, $c \in P(d)$. Contrapositively, there is no sensitive column $c \in \mathcal{S}$ with $c \notin P(d)$ appearing in any C_i —i.e., no such c appears anywhere in a ’s derivation. Since Assumption 1 guarantees $a.\text{lineage}$ records every column touched by the computation

that produced $a.\text{value}$, this is precisely the safety property claimed for the value returned at line 10.

Case 2: fallback path (line 14). The input precondition to Algorithm 1 (stated in the signature) requires that *fresh* is computed entirely within $P(d)$ by the caller, before Algorithm 1 is invoked. Under Assumption 1, *fresh.lineage* records every column of that computation, so every column recorded is, by the precondition, in $P(d)$; in particular no sensitive column outside $P(d)$ is touched.

Both cases establish that the AMU returned to department d has a derivation containing no sensitive column outside $P(d)$, for an arbitrary invocation with arbitrary *store* contents. Since the two cases are exhaustive over all return statements, the claim holds for every invocation of Algorithm 1. $\square$

Corollary 2 (Robustness to unsafe store contents). *The guarantee of Theorem 1 holds even when store contains AMUs whose derivation paths touch sensitive columns $c \in \mathcal{S}$ with $c \notin P(d)$: such candidates fail the guard at line 6 for department d ’s request and are skipped, never returned, regardless of how many such unsafe candidates exist or where they occur in candidates. This matters operationally because a shared store necessarily contains AMUs written by every department, including ones whose lineage is unsafe for the department currently requesting: the guarantee does not depend on the store being curated per department.*

Corollary 3 (Behaviour under policy updates). *Let $P_1(d)$ and $P_2(d)$ be two successive versions of department d ’s permission set, and let $\mathcal{S}_1, \mathcal{S}_2$ be two successive versions of the sensitive-column registry. If $S(a)$ is cached using $\mathcal{S}_1$ at write time and the organisation transitions to $(\mathcal{S}_2, P_2)$ without recomputing $S(a)$ for existing AMUs, Theorem 1’s guarantee holds only with respect to $(\mathcal{S}_1, P_1)$, not $(\mathcal{S}_2, P_2)$: a column newly added to $\mathcal{S}_2$ that appears in a ’s lineage but was not in $\mathcal{S}_1$ will not appear in the stale cached $S(a)$, and a candidate that should now be blocked may instead pass the gate. The guarantee is restored for all subsequent retrievals as soon as $S(a)$ is recomputed against $\mathcal{S}_2$ for every AMU with a write epoch preceding the policy change—an $O(N)$ one-time cost over the N AMUs in the store, after which each retrieval again costs $O(n)$ as in Theorem 1. In practice we recommend the lazy recomputation strategy of the implementation considerations (main paper, §3): recompute $S(a)$ the first time an AMU is touched after a policy-version bump, amortising the $O(N)$ cost over subsequent retrievals rather than paying it eagerly. This corollary formalises threat T3 (the threat model (main paper, §5)).*

Remark (Assumption dependency). *Assumption 1 is critical. If lineage is reported by the computing agent rather than extracted automatically from query execution plans, a buggy or adversarial agent can omit a column, breaking the guarantee. Automatic extraction via systems such as LINEAGEX [12] is the appropriate production countermeasure; this dependency is also catalogued as threat T5 in the threat model (main paper, §5). The completeness-degradation experiment (main paper, §6) empirically quantifies how the guarantee degrades as a function of lineage completeness.*

IV. EXTENDED THREAT DESCRIPTIONS

The main paper (§5, Table 4) summarises all eight threats in tabular form. The full prose description of each threat, reproduced from the original threat-model analysis, follows.

A. Threats Addressed

B. Threats Addressed

T1 — Sensitive-column leakage via cached result. An agent retrieves a result derived from a sensitive column $c \in \mathcal{S}$ it is not

permitted to see. *Mitigation:* Algorithm 1 blocks such retrievals via the $S(a) \subseteq P(d)$ gate. Non-sensitive columns outside $P(d)$ are not gate-enforced by the current design; see Remark 3.

T2 — Silent metric definition drift. Two teams store conflicting definitions of the same KPI under the same name. *Mitigation:* Algorithm 2 surfaces the divergence on write via the definition-hash conflict detector.

T3 — Stale sensitive data reuse. An AMU derived from data that was sensitive at write time is retrieved after column permissions are updated. *Mitigation:* the `epoch` field enables TTL-based eviction; sensitivity tags should be re-evaluated on retrieval when policies change.

C. Threats Not Addressed

D. Threats Not Addressed

T4 — Inference attacks. A derived feature that *encodes* sensitive information without naming the source column (e.g., a risk score computed from `income`) will not be caught unless the dependency is explicitly recorded in the lineage graph. This is the most important production gap.

T5 — Malicious lineage fabrication. A rogue agent writes an AMU with an intentionally incomplete lineage graph, bypassing the gate. The degradation experiment (the completeness-degradation experiment (main paper, §6)) quantifies how even *accidental* under-reporting erodes the guarantee. Automatic lineage extraction is the appropriate countermeasure.

T6 — Aggregate inference. A sequence of individually safe retrievals may allow reconstruction of sensitive values. Algorithm 1 is a per-retrieval check; it provides no differential privacy guarantee.

T7 — Insider threats. An operator with direct store access can read or modify entries. The mechanism assumes a trusted store layer.

T8 — Prompt-injection and timing attacks on the lineage record.

Two related attack surfaces are specific to the lineage layer itself and are not covered by Theorem 1. First, an indirect prompt injection embedded in tool output or retrieved content could manipulate an LLM-orchestrated agent into issuing a query that omits a sensitive join from its *self-reported* lineage, or into requesting that a differently derived (but identically named) result be written under a misleading `metric_name`; this is a lineage-specific instance of the broader agent attack surface catalogued by [16–19]. Second, differential timing of the retrieve path itself is a theoretical side channel: because the worst-case branch (all candidates blocked, $O(n)$) and the best-case branch (first candidate safe, $O(1)$) have measurably different latencies (Table 6), a co-located observer could in principle infer whether *some* cross-department AMU exists for a given metric name, even without learning its value—an inference analogous to the membership-inference attacks demonstrated against retrieval-augmented generation systems by Naseh et al. [23]. *Mitigation:* for the prompt-injection surface, lineage should be extracted automatically from the executed query (as in Experiment 6, the real-agent experiment (main paper, §6)) rather than self-reported by the agent, and defences such as AuthGraph’s provenance/authorisation cross-check [22] are complementary and can run upstream of the AMU write path. For the timing surface, constant-time gate evaluation (padding every retrieval to worst-case latency) closes the channel at a fixed

$13.8\mu\text{s}$ -per-retrieval cost, which Table 6 shows is negligible relative to any real analytics query; we do not implement this hardening in the current prototype and flag it as required only for deployments with an explicit timing-side-channel threat in scope.

V. PROPOSED COMPOSITE CONFLICT DETECTOR (D4, UNEVALUATED)

The main paper’s extended fuzzy conflict-detection experiment (§6) reports that the column-graph detector D3 is blind to logic-operator changes (the “LO blind spot”). Motivated by this finding, we sketch a composite detector D4 that combines D3’s column-graph comparison with an explicit structural check on filter logic. **D4 has not been implemented or evaluated;** it is a design proposal reported as future work, precisely so it can be benchmarked against the same 43-pair dataset before any production claim is made about it. The design intent is that D4 subsumes D3 exactly on True-Conflict and Threshold-Variant pairs (where column-graph comparison alone is already correct) while adding sensitivity to Logic-Operator pairs by comparing the *operator multiset* of the parsed filter expression rather than its literal token similarity.

Algorithm 3: Proposed Composite Conflict Detector (D4, un-evaluated) Input: lineage pair (L_1, L_2) Output: *conflict bool*

```

1  % L1, L2 : two lineage graphs for the same
   % metric_name from different depts
2  % Combines D3 (column-graph) with an explicit
   % filter-logic structural check.
3  % NOT implemented or evaluated in this paper --
   % design proposal only.
4
5  tables1, cols1 <- tables_and_columns(L1)
6  tables2, cols2 <- tables_and_columns(L2)
7
8  if tables1 != tables2 or cols1 != cols2:
9      return True % D3's
   % existing check: structural join/column diff
10
11 % Column-graphs are identical; D3 alone would
   % return False here.
12 % D4 adds an explicit comparison of filter-logic
   % structure.
13 ast1 <- parse_filter_logic(L1.varphi) % e.g. via a
   % boolean-expression parser
14 ast2 <- parse_filter_logic(L2.varphi)
15
16 if operator_multiset(ast1) != operator_multiset(
   ast2):
17     return True % AND/OR/NOT
   % composition differs: logic-operator conflict
18
19 if threshold_literals(ast1) != threshold_literals(
   ast2):
20     return False % same logical
   % structure, only a numeric threshold differs
21 % (TV case:
   % not a
   % conflict
   % ,
   % matches
   % D3's
   % original
   % design
   % intent)
22
23 return False % identical
   % column-graph and identical filter-logic
   % structure

```

VI. COMPLETE PER-EXPERIMENT RESULT TABLES

The main paper (§6) reports condensed results tables. The complete, un-condensed tables for every experiment are reproduced below.

- A. *Experiment 1: Department Permissions and Full Results*
- B. *Experiment 2: TPC-H Full Results*
- C. *Experiment 3: Full Degradation Table*
- D. *Experiment 4: Full Fuzzy Conflict-Detection Results*
- E. *Experiment 5: Complete Runtime Measurements*
- F. *Experiment 6: All Real-Agent Round-Trips*

REFERENCES

- [1] P. Du, “Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers,” *arXiv preprint arXiv:2603.07670*, 2026.
- [2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2310.08560.
- [3] P. Rasmussen, “Zep: A temporal knowledge graph architecture for agent memory,” *arXiv preprint arXiv:2501.13956*, 2025.
- [4] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.12110.
- [5] H. Taheri, “Governed memory: A production architecture for multi-agent workflows,” *arXiv preprint arXiv:2603.17787*, 2026.
- [6] C. Lam, J. Li, L. Zhang, and K. Zhao, “Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework,” *arXiv preprint arXiv:2603.11768*, 2026.
- [7] Oracle Corporation, “Oracle AI Agent Memory: A governed, unified memory core for enterprise AI agents.” Oracle Developer Blog, March 2026.
- [8] N. Abaev, D. Klimov, G. Levinov, D. Mimran, Y. Elovici, and A. Shabtai, “AgentGuardian: Learning access control policies to govern AI agent behavior,” *arXiv preprint arXiv:2601.10440*, 2026.
- [9] P. Subramaniam and S. Krishnan, “DePLOI: Applying NL2SQL to synthesize and audit database access control,” *arXiv preprint arXiv:2402.07332*, 2024.
- [10] L. Moreau, B. Clifford, J. Freire, J. Futrelle, Y. Gil, P. Groth, N. Kwasnikowska, S. Miles, P. Missier, J. Myers, B. Plale, Y. Simmhan, E. Stephan, and J. V. den Bussche, “The open provenance model — core specification (v1.1),” *Future Generation Computer Systems*, vol. 27, no. 6, pp. 743–756, 2011. See also W3C PROV-DM: <https://www.w3.org/TR/prov-dm/>.
- [11] M. Herschel, R. Diestelkämper, and H. Ben Lahmar, “A survey on provenance: What for? what form? what from?,” *The VLDB Journal*, vol. 26, no. 6, pp. 881–906, 2017.
- [12] S. H. Zhang, Z. Miao, and J. Wang, “LINEAGEX: A column lineage extraction system for SQL,” in *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE), Demo Track*, 2025. arXiv:2505.23133.
- [13] X. Zhou, J. He, W. Zhou, H. Chen, Z. Tang, H. Zhao, X. Tong, G. Li, Y. Chen, J. Zhou, Z. Sun, B. Hui, S. Wang, C. He, Z. Liu, J. Zhou, and F. Wu, “A survey of LLM × DATA,” *arXiv preprint arXiv:2505.18458*, 2025.
- [14] T. Mao and contributors, “sqlglot: SQL parser and transpiler.” <https://github.com/tobymao/sqlglot>, 2023. MIT Licence. Version 23+ used in this work.
- [15] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, “AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems,” *arXiv preprint arXiv:2602.11510*, 2026.
- [16] A. Naik, J. Culligan, Y. Gal, P. Torr, R. Aljundi, A. Paren, and A. Bibi, “OMNI-LEAK: Orchestrator multi-agent network induced data leakage,” *arXiv preprint arXiv:2602.13477*, 2026.
- [17] M. Alizadeh, Z. Samei, D. Stetsenko, and F. Gilardi, “Simple prompt injection attacks can leak personal data observed by LLM agents during task execution,” *arXiv preprint arXiv:2506.01055*, 2025.
- [18] P. Wang, X. Li, C. Xiang, J. Zhang, Y. Li, L. Zhang, X. Wang, and Y. Tian, “The landscape of prompt injection threats in LLM agents: From taxonomy to analysis,” *arXiv preprint arXiv:2602.10453*, 2026.
- [19] A. Dehghantanha and S. Homayoun, “SoK: The attack surface of agentic AI — tools, and autonomy,” *arXiv preprint arXiv:2603.22928*, 2026.
- [20] T. Nguyen, M. Ndebugre, and D. Arremsetty, “Security considerations for multi-agent systems,” *arXiv preprint arXiv:2603.09002*, 2026.
- [21] C. Summers and E. Wu, “Data flow control: Data safety policies for AI agents,” *arXiv preprint arXiv:2606.05679*, 2026.
- [22] P. Wang, Y. Li, and Y. Tian, “Aligning provenance with authorization: A dual-graph defense for LLM agents,” *arXiv preprint arXiv:2605.26497*, 2026.
- [23] A. Naseh, Y. Peng, A. Suri, H. Chaudhari, A. Oprea, and A. Houmansadr, “Riddle me this! stealthy membership inference for retrieval-augmented generation,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025. arXiv:2502.00306.
- [24] Colrows, “Why BI metrics do not match across dashboards.” Technical Blog, 2024.
- [25] Snowflake, Salesforce, dbt Labs, BlackRock, and RelationalAI, “Open semantic interchange (OSI): A vendor-neutral semantic layer specification for AI and BI.” Industry Standard Initiative, September 2025.

Table 1. Department permissions (synthetic schema). The gate (Algorithm 1) enforces restrictions only for columns in the sensitive registry $\mathcal{S} = \{\text{ssn}, \text{email}, \text{income}\}$; non-sensitive blocked columns (`amount`, `spend`) represent the full policy intent but are not currently gate-enforced (see Remark 3).

Department	Permitted (summary)	Blocked (gate-enforced in $\mathcal{S}$)
Finance	All columns	None
Marketing	Commercial, order data	<code>ssn</code> , <code>income</code> , <code>email</code>
Support	Customer, ticket meta	<code>ssn</code> , <code>income</code> , <code>email</code> (<code>amount</code> , <code>spend</code> : policy only)

Table 2. Synthetic schema results (30 seeds, mean $\pm$ population std dev). Bootstrap 95% CI on naive–LA difference excludes zero for all comparisons (Mann-Whitney U , $p < 0.001$).

Metric	No Memory	Naive Shared	Lineage-Aware	Diff CI (95%)
Leak rate (%)	0.0 ± 0.0	18.8 ± 3.7	$0.0 \pm 0.0^\dagger$	$[+14.7, +22.8]$
Memory reuse rate (%)	0.0 ± 0.0	95.8 ± 0.0	82.6 ± 2.3	$[+11.2, +15.1]$
Conflict recall (%)	n/a	0.0 ± 0.0	100.0 ± 0.0	—
Blocked retrievals (seed=42)	0	0	15	—

[†]Formal guarantee (Theorem 1), not an empirical discovery.**Table 3.** TPC-H-inspired schema results (30 seeds, mean $\pm$ population std dev). Both comparisons: $p < 0.001$, bootstrap 95% CI excludes zero.

Metric	Naive Shared	Lineage-Aware	Diff CI (95%)
Leak rate (%)	25.5 ± 4.5	$0.0 \pm 0.0^\dagger$	$[+19.7, +31.2]$
Memory reuse rate (%)	95.8 ± 0.0	81.5 ± 4.8	$[+10.4, +18.2]$
Metric-conflict recall (%)	0.0 ± 0.0	100.0 ± 0.0	—

[†]Formal guarantee (Theorem 1).**Table 4.** Lineage-completeness degradation: leak rate vs. reporting completeness ρ (30 seeds each, mean $\pm$ population std dev). At $\rho=1.0$ the guarantee holds by Theorem 1.

Schema	$\rho = 0.50$	$\rho = 0.75$	$\rho = 0.90$	$\rho = 1.00$
Synthetic (5-table)	$16.3 \pm 3.3\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$
TPC-H (8-table)	$10.3 \pm 4.7\%$	$0.9 \pm 1.1\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$

Table 5. Extended fuzzy conflict-detection results (43 pairs: 15 TC + 15 TV + 8 LO + 5 CS). Bootstrap 95% CI on F_1 shown in square brackets. LO column shows per-category TP/total detection.

Detector	Prec.	Recall	F_1	95% CI (F_1)	LO correct
D1 — Exact hash	0.651	1.000	0.789	$[0.677, 0.883]$	8/8 ✓
D2 — Jaccard ($\tau=0.70$)	0.676	0.821	0.742	$[0.604, 0.853]$	3/8
D3 — Column-graph	1.000	0.714	0.833	$[0.703, 0.933]$	0/8 ✗

Table 6. Empirical runtime (mean $\pm$ std dev over 2,000 iterations). $k=2$ –3 lineage steps, $c=4$ –7 columns per step.

Operation	n	Mean (μs)	Std dev (μs)
Gate-check predicate (set-difference)	any	0.09	0.03
Sensitivity-tag derivation ($O(kc)$)	—	0.35	0.13
Definition-hash computation ($O(kc)$)	—	1.20	0.46
Retrieve, best-case (first hit safe)	1	0.66	0.23
Retrieve, best-case (first hit safe)	50	0.64	0.22
Retrieve, worst-case (all blocked)	1	0.66	0.12
Retrieve, worst-case (all blocked)	5	1.74	0.22
Retrieve, worst-case (all blocked)	10	3.07	0.39
Retrieve, worst-case (all blocked)	20	5.72	0.45
Retrieve, worst-case (all blocked)	50	13.82	0.76
Write, no-conflict (full scan, $O(n)$)	1	0.28	0.03
Write, no-conflict (full scan, $O(n)$)	50	1.63	0.20
Write, conflict detected (early exit)	1	2.37	1.12
Write, conflict detected (early exit)	50	2.33	0.61

Table 7. Experiment 6: all 9 agent round-trips. “REUSE” = served from cached AMU; “WRITE” = fresh SQL executed and AMU written; “BLOCK” = gate blocked at least one candidate before falling back to a fresh computation.

Label	Dept	Metric	Action	Blocked	Conflict	Leaked
A1	Finance	revenue_by_category	WRITE	No	No	No
A2	Sales	revenue_by_category	REUSE	No	No	No
A3	Operations	revenue_by_category	WRITE	Yes	Yes	No
B1	HR	employee_directory	WRITE	No	No	No
B2	Finance	employee_directory	REUSE	No	No	No
B3	Sales	employee_directory	WRITE	Yes	Yes	No
C1	Finance	orders_by_country	WRITE	No	No	No
C2	Operations	orders_by_country	REUSE	No	No	No
C3	Sales	orders_by_country	REUSE	No	No	No
<i>Totals (9 round-trips)</i>				2 blocked	2 conflicts	0 leaked